

Building the Trivia Game

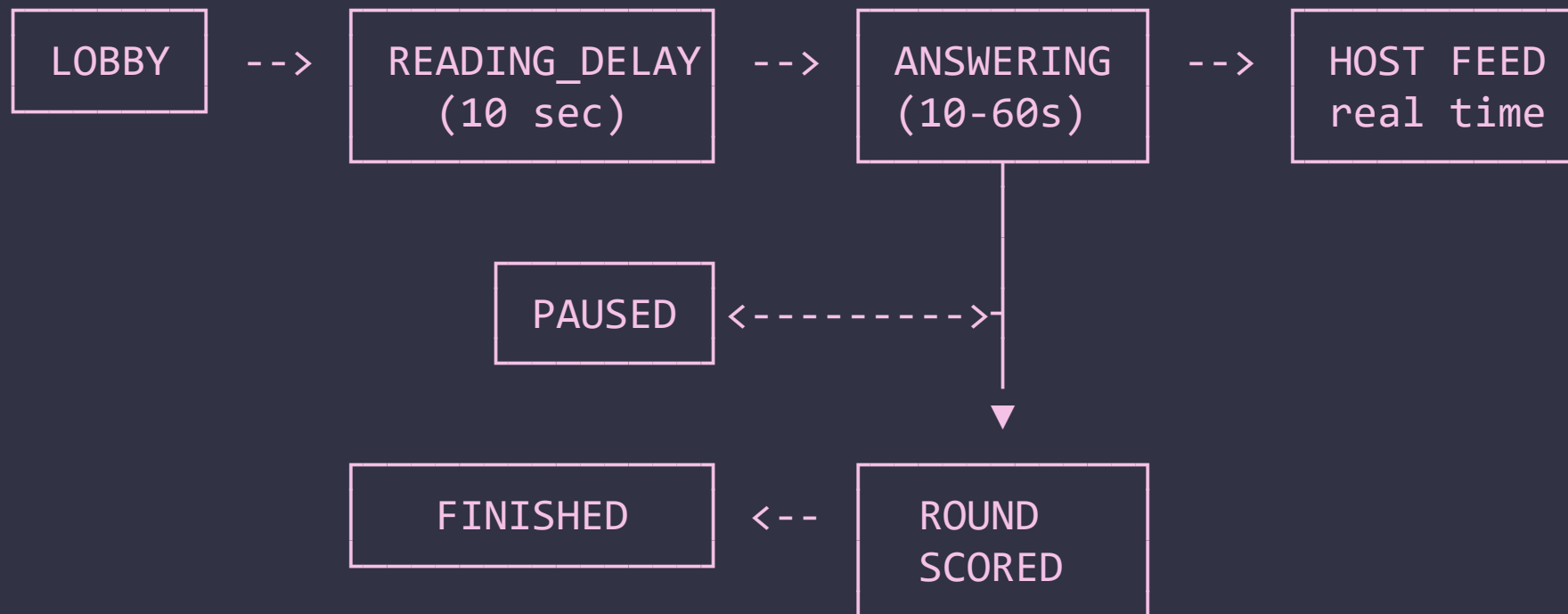
How did I do something I had no business doing in like 8 hours?

A real-time multiplayer trivia platform, architecture & design overview

What are some of the Game Features 🎮

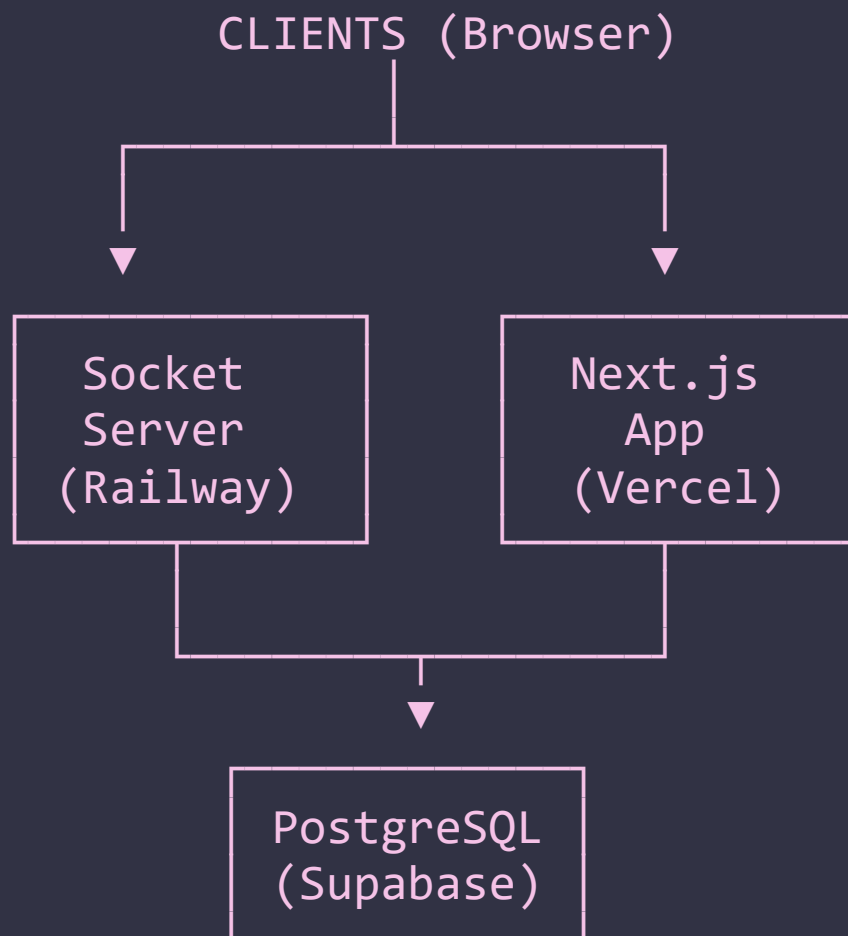
Feature	Implementation
Fully configurable games	Questions & answers loaded with a CSV, timer at launch
Hardest Questions	Auto-calculates lowest correct rate & displays
Live leaderboard	Updates after each round
Separate host/player clients	Different UI/UX for each
Response time tracking	Potential for tiebreakers
Team names	Customizable per team
Completely web-based	No downloads, works on any device
Guest access	No auth/signup needed, easy QR code

Game Flow State Machine





System Architecture



```
// Next.js 14 + react 18 + TypeScript  
// Tailwind CSS for styling  
// Socket.IO Client for real-time comms
```

```
// When new features/bugs are pushed:  
// <--- Fed by Github <--- Fed by VSCode
```

🧑 What are these components: Vercel

- Cloud-based software to deploy and host modern frontend frameworks and static websites with easy config.
- Provided great feedback on compiler issues when setting up.
- Allowed for multiple environment settings (via config parameters).
- Has a lot of monitoring / environment scaffolding.

The screenshot shows the Vercel dashboard for a project named "trivia-game-project-5wwn". The top navigation bar includes links for Overview, Deployments, Analytics, Speed Insights, Logs, Observability, Redirects, Firewall, AI Gateway, Storage, Flags, and Settings. The project name is displayed prominently, along with buttons for Repository, Usage, Domains, and a Visit button. Below this, the "Production Deployment" section is active, showing a preview of the "Trivia Game" application. The deployment details indicate it is "Ready" and was created 2 days ago by "scschwa". The source is identified as "main" with a commit hash "b4d226d". A note mentions adding a configurable answering time to the game setup. The "Deployment Settings" section includes recommendations for building multiple deployments simultaneously, getting builds up to 40% faster, preventing frontend-backend mismatches, and finding a custom domain. The "Build Settings" section shows "On-Demand Concurrent Builds" as disabled, "Build Machine" as Standard with 4 vCPUs and 8 GB Memory, and "Prioritize Production Builds" as enabled. The "Runtime Settings" section shows "Fluid Compute" as enabled, "Function CPU" as Standard with 1 vCPU and 2 GB Memory, "Node.js Version" as 24.x, "Deployment Protection" as Standard Protection, "Skew Protection" as disabled, and "Cold Start Prevention" as disabled.

Stephen Schwartz's projects Hobby / trivia-game-project-5wwn

Find... Feedback

Overview Deployments Analytics Speed Insights Logs Observability Redirects Firewall AI Gateway Storage Flags Settings

trivia-game-project-5wwn Repository Usage Domains Visit

Production Deployment Build Logs Runtime Logs Instant Rollback

Trivia Game
Host and play live code games with friends

Join a Game
Enter Game Code
ABCEDEF
Join Game

Host a Game
Create New Trivia Config
Host a Game Now

View Your Games & Settings

Deployment
trivia-game-project-5wwn-gkvojwg44-stephen-schwartzs-projects.vercel.app

Domains
trivia-game-project-5wwn.vercel.app

Status
Created
Ready 2d ago by scschwa

Source
main
b4d226d Add configurable answering time (10-60 seconds) to game setup

Deployment Settings

Recommendations

Build Multiple Deployments Simultaneously
Never wait for a queued build.

Get builds up to 40% faster
Switch to a bigger build machine.

Prevent Frontend-Backend Mismatches
Automatically sync client and server versions to avoid deployment conflicts.

Find a Custom Domain
Purchase a domain. Fast, at-cost & private.

Build Settings

On-Demand Concurrent Builds
Disabled

Build Machine
Standard 4 vCPUs 8 GB Memory

Prioritize Production Builds
Enabled

Runtime Settings

Fluid Compute
Enabled

Function CPU
Standard 1 vCPU 2 GB Memory

Node.js Version
24.x

Deployment Protection
Standard Protection

Skew Protection
Disabled

Cold Start Prevention
Disabled

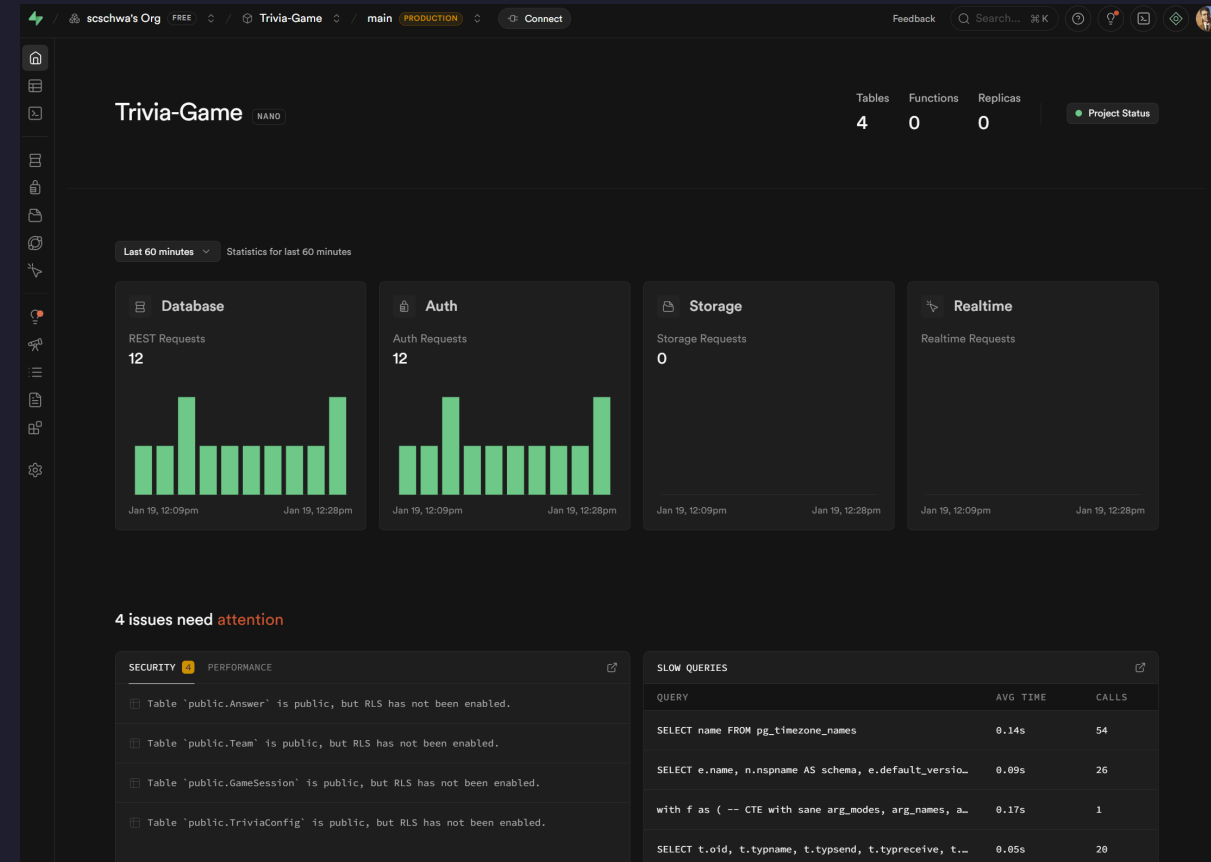
🧑 What are these components: Railway.app

- Only a portion of the project is deployed here (socket-server).
- A back-end equivalent of Vercel. Allows for auto-scaling with each inbound request of the socket server.
- Database connectivity is pooled, with direct multi-client write/read enabled.
- \$5 spend if you have an older github account to link to it.

The screenshot shows the Railway.app dashboard for a project named 'trivia-game-project'. The top navigation bar includes the project name, environment ('production'), and various tabs like 'Architecture', 'Observability', 'Logs', and 'Settings'. A status bar at the top right indicates '24 days or \$4.90 left'. The main content area is divided into two panels. The left panel shows a grid of service icons, with one icon for 'trivia-game-project' labeled 'trivia-game-project-producti...' and 'Online'. The right panel is titled 'trivia-game-project' and contains tabs for 'Deployments', 'Variables', 'Metrics', and 'Settings'. The 'Deployments' tab is active, showing a list of deployments. The first deployment is 'trivia-game-project-production.up.railway.app', which is 'ACTIVE' and was deployed '2 days ago via GitHub'. It shows a 'Deployment successful' status and a list of steps: 'Initialization' (00:00), 'Build' (00:25), 'Deploy' (00:05), and 'Post-deploy' (00:00). Below this, a 'HISTORY' section lists several other deployments, all marked as 'REMOVED', including 'Remove correct answer reveal on timer e...', 'Add Scoring Round loading popup', 'Add Hardest Questions section to final sc...', 'Remove Score Comparison chart from le...', and 'Add final trivia questions - 7 rounds cove...'. Each deployment entry includes a 'View logs' link and a three-dot menu icon.

🧑 What are these components: Supabase

- Open-source database-as-a-service (DaaS, but the good kind, not like CDA-EU 😊).
- This lets me build, host, and connect a PostgreSQL database with minimal setup.
- It also provides the appropriate connection pooling needed for serverless environments like Vercel/Railway.
- Has a really nice dashboard for running queries, inspecting data, and managing roles.



Why This Architecture?

Decision	Rationale
Separate Socket Server	Vercel doesn't support persistent WebSockets
Server-Authoritative Timers	Ensures sync with dynamic offsets (was having issues w/ this)
PostgreSQL	Required for serverless (Supabase pooling features were clutch)
Next.js Server Actions	Type-safe mutations, showing any potential errors upfront, this got a lot of debugging out of the way.
Prisma ORM	Shared schema, but type-safe DB to avoid problems/conflicts that could crash clients unexpectedly, ruining gameplay



Lines of Code by Language

Language	Lines	Files	Purpose
TypeScript/TSX	7,819	46	Application logic, React components
CSS	4,846	2+	Styling (Tailwind + custom)
Prisma Schema	~290	2	Database models
JSON Config	327	6	Package & TypeScript config
Documentation	709	4+	README, guides
Total	~14,000	~60	

I wrote **basically zero** of it (besides the game questions and this presentation).

Frontend Dependencies (18 packages)

Category	Packages
Framework	<code>next</code> (14.1), <code>react</code> (18.2), <code>react-dom</code>
Database	<code>@prisma/client</code> (5.10)
UI Components	<code>lucide-react</code> , <code>recharts</code> , <code>qrcode.react</code>
Forms	<code>react-hook-form</code> , <code>zod</code>
File Handling	<code>papaparse</code> (CSV), <code>react-dropzone</code>
Real-time	<code>socket.io-client</code>
Auth	<code>bcrypt</code> , <code>jose</code> (JWT)

I couldn't have navigated this without significant tutorials. I had never used `npm install` or `npx prisma *` before starting this project.

Socket Server Dependencies (8 packages)

Category	Packages
Server	<code>express</code> , <code>socket.io</code> , <code>cors</code>
Database	<code>@prisma/client</code>
Auth	<code>bcrypt</code> , <code>jose</code>
Config	<code>dotenv</code>
Validation	<code>zod</code>

Ditto above - I had never built a socket server before, so this was all new to me.

Technical Highlights

1. Clock Skew Compensation

- `serverTime` offset syncs timers across devices

2. Reconnection Support

- Unique `reconnectToken` in localStorage

3. Archive vs Delete

- Used configs archived (data integrity)
- Unused configs deletable

4. QR Code Joining

- Easy mobile player onboarding

5. Keyboard Shortcuts

- Host controls: Space, P, N, S, L, Esc

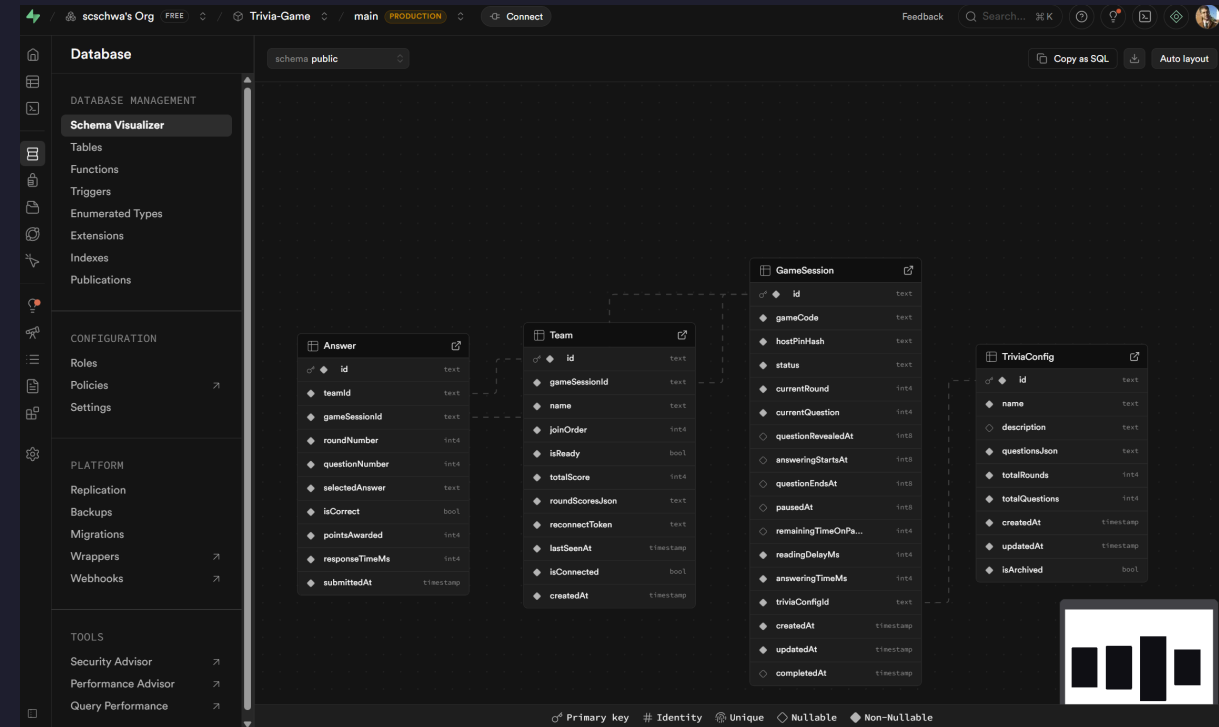


Database Design

- Questions as JSON blob - Faster reads, simpler schema
- Per-round scores - Enables breakdown analytics
- Strategic indexes - gameCode, status, createdAt

4 Table Models:

TriviaConfig → GameSession → Team → Answer



Multi-Device Experience

Host/Presenter

- **Host:** Large display with game controls assuming projection.
- **Players:** PC, Tablet, or Mobile-optimized answer buttons/iinterface.
- **Scoreboard:** Dedicated final results display.

Security Measures

-  Host PIN hashed with **bcrypt** for secure play
-  Unique team **reconnect tokens** for session integrity
-  Rate limiting: **5 events/sec** per socket avoids spamming
-  Input validation with **Zod schemas** which prevents data issues

Development Statistics

Metric	Value
Total Commits	33
Development Period	5 days
Lines Inserted	23,599
Lines Deleted	264
Net Lines Added	~23,335
Average Commits/Day	6.6

Most of this was up and running in about 2-3 hours from the initial prompt. I spent another 4-5 hours debugging, polishing, and adding features, and doing some playthrus on multiple clients.

Commit Breakdown

Category	Count	%
Bug Fixes	18	55%
Features	6	18%
Infrastructure	4	12%
Documentation	3	9%
Other	2	6%

Almost all commits were small and focused, with the majority being performed by Claude Opus 4.5.



Traditional Development Estimate

How long should this have taken?

Component	Junior	Mid-Level	Senior
Database schema & Prisma setup	2 days	1 day	0.5 days
Next.js frontend (pages, components)	5-7 days	3-4 days	2 days
Socket server (real-time logic)	4-5 days	2-3 days	1-2 days
Timer synchronization system	2-3 days	1-2 days	1 day
Auth & security (PIN, tokens)	2 days	1 day	0.5 days
CSV upload & validation	1-2 days	1 day	0.5 days
Deployment & debugging	3-4 days	2 days	1 day
UI polish & styling	3-4 days	2 days	1-2 days
Total	4-6 weeks	2-3 weeks	1-2 weeks

Development Time Comparison

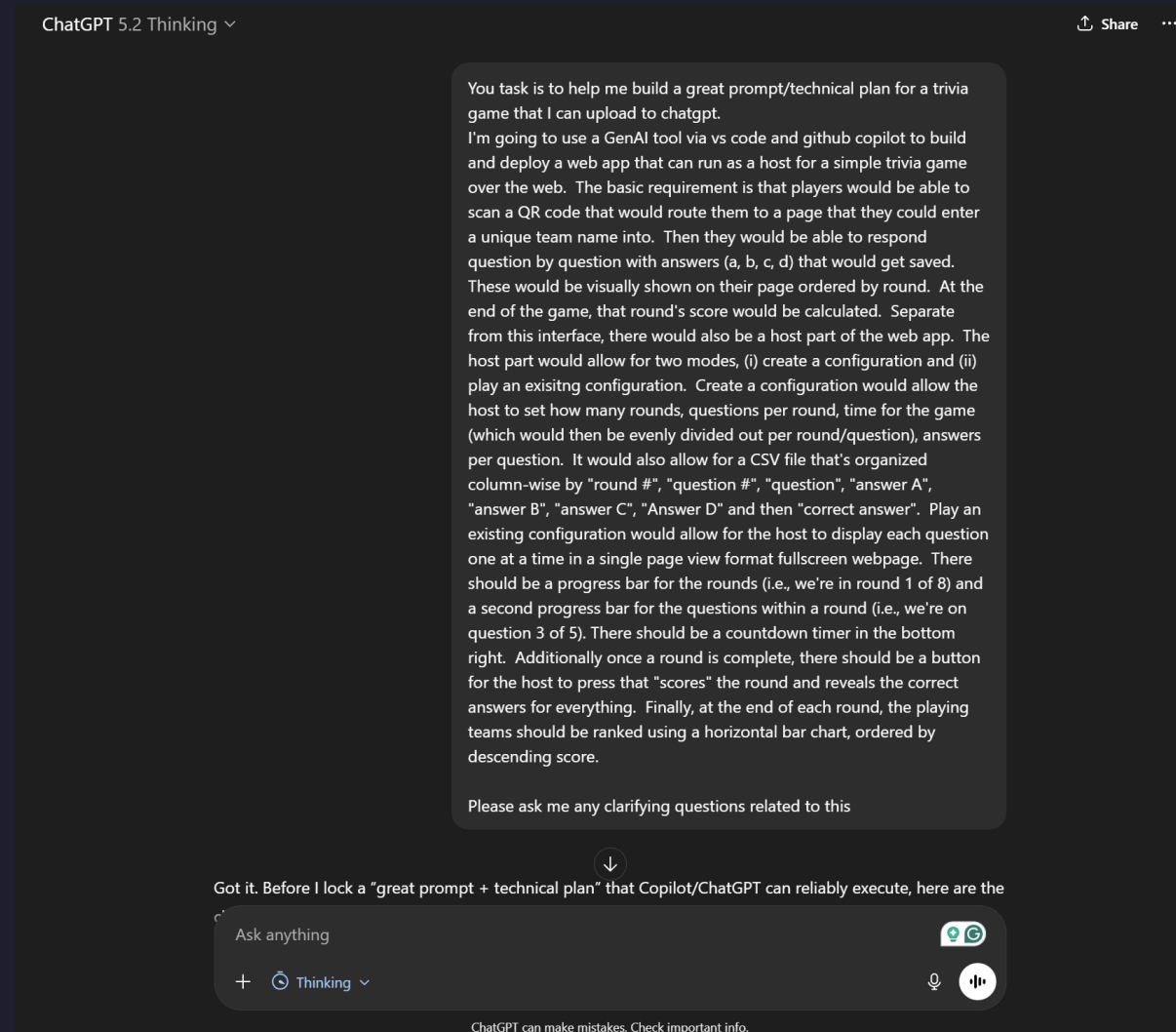
🕒 Development Time: AI-Assisted

Metric	Value
Actual time spent	~8 hours
Calendar span	5 days
Lines of code	~14,000
Lines per hour	~1,750

~10-20x speedup with GHCP and Claude 4.5 Opus AI vs. a Mid-level / Senior Developer.

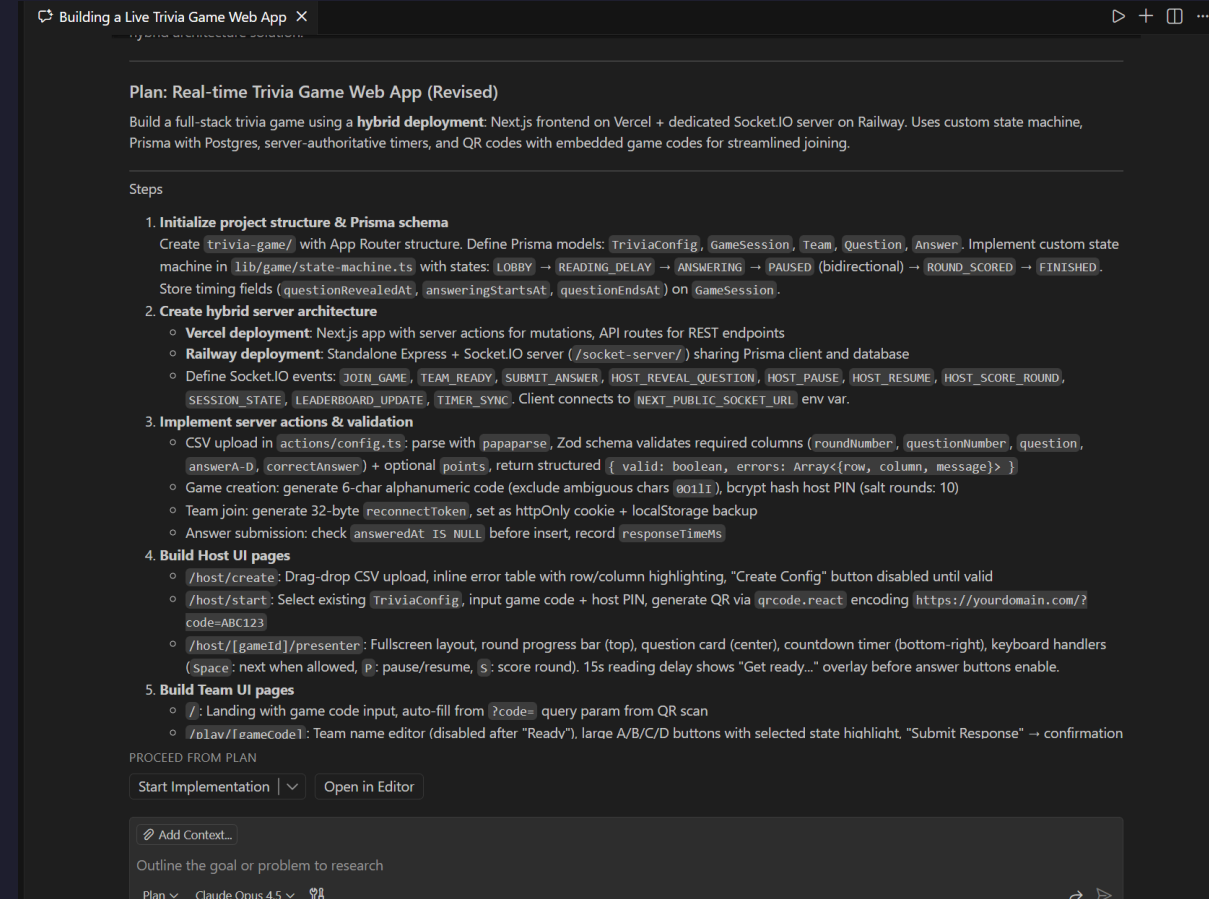
What was my workflow?

- I started with ChatGPT, 5.2 Thinking with this prompt.
- It asked 23 follow-up questions to clarify requirements, which I provided.
- This generated a markdown-based implementation spec, which I could easily pass into GHCP via VSCode.



What was my workflow? (con't)

- I am using VSCode 1.108.0 (December 2025) with GHCP Copilot extension.
- I made a new project folder, launch VSCode, dropped in this imp-spec and asked GHCP in `planning` mode to create an imp-plan.
- Once satisfied and done with questions, I asked it to implement.



What was my workflow? (con't)

- Building took a good bit of time and hit context cap a few times.
- This required me to manually advance it during those (~5-8 minutes or so).
- Once done, I requested a deployment guide to test it on local desktop.
- This did take a bit of tweaking to get right (local and cloud).

review DEPLOYMENT_GUIDE.md X

Trivia Game - Complete Deployment Guide

This guide walks you through deploying the Trivia Game application from scratch, including setting up the database, deploying the servers, and configuring your GoDaddy domain.

Table of Contents

1. Prerequisites
2. Local Development Setup
3. Production Database (Supabase)
4. Deploy Socket.IO Server (Railway)
5. Deploy Next.js App (Vercel)
6. Configure GoDaddy Domain
7. Final Testing
8. Maintenance & Updates

Prerequisites

Before starting, ensure you have:

- **Node.js 18+** installed ([download](#))
- **Git** installed ([download](#))
- **GitHub account** (for deployment integrations)
- **GoDaddy account** with a domain you want to use
- **Credit card** (Railway requires payment info, but has a free tier)

Local Development Setup

Step 1: Clone and Install

```
# Clone the repository (or navigate to your existing folder)
cd trivia-game-project

# Install main app dependencies
npm install

# Install socket server dependencies
cd socket-server
npm install
cd ..
```

Step 2: Create Environment Files

Main app - Create `.env.local` in the project root:

What was my workflow? (con't)

See this in action

The screenshot displays a developer's workflow in a multi-pane environment. On the left, the GitHub web interface for the 'trivia-game-project' repository is visible, showing a list of files and a commit history. The central pane shows a code editor with the 'page.jsx' file open, displaying JavaScript code for handling game state, including functions for loading configurations, deleting trivia questions, and archiving them. The right pane shows a terminal window with the output of a build command, indicating a successful build and deployment. The terminal output includes file sizes and timestamps for various assets like 'package.json', 'postcss.config.js', 'README.html', 'README.md', 'tailwind.config.js', 'trivia_plan.md.txt', and 'tsconfig.json'.

trivia-game-project

src > app > host > start > @ page.jsx ...

```
22 export default function StartGamePage() {
23   const [showArchived, setShowArchived] = useState(false);
24   useEffect(() => {
25     loadConfigs();
26   }, [showArchived]);
27   const loadConfigs = async () => {
28     const data = await getTriviaConfigs(showArchived);
29     setConfigs(data);
30   };
31   // Only auto-select if we don't have a selection and there are non-archived
32   const activeConfigs = data.filter(c => !c.isArchived);
33   if (activeConfigs.length > 0 && !selectedConfigId) {
34     setSelectedConfigId(activeConfigs[0].id);
35   }
36   setisLoading(false);
37 }
38
39 const handleDelete = async (id: string) => {
40   const result = await deleteTriviaConfig(id);
41   if (result.success) {
42     await loadConfigs();
43   } else {
44     setError(result.error || 'Failed to delete');
45   }
46   setDeleteConfirmId(null);
47 }
48
49 const handleArchive = async (id: string, archive: boolean) => {
50   const result = await archiveTriviaConfig(id, archive);
51   if (result.success) {
52     if (selectedConfigId === id && archive) {
53       setSelectedConfigId('');
54     }
55     await loadConfigs();
56   } else {
57     setError(result.error || 'Failed to archive');
58   }
59   setArchiveConfirmId(null);
60 }
61
62 const handleReady = () => {
63   const result = await archiveTriviaConfig(selectedConfigId, true);
64   if (result.success) {
65     setSelectedConfigId('');
66   }
67   await loadConfigs();
68 }
69
70 setError(result.error || 'Failed to archive');
71
72 setArchiveConfirmId(null);
73
74 }
```

PS C:\Users\janev\OneDrive\trivia-game-project> dir

	1/12/2026 10:02 PM	1498
package.json		
postcss.config.js	89	
README.html	39448	
README.md	2234	
tailwind.config.js	2135	
Test_question_set.csv	325	
trivia_plan.md.txt	7827	
tsconfig.json	621	

Without changing code, could you check to see how difficult it would be to either (a) add 3 seconds to the transition between the check box appearing and the "Get Ready" screen effect, (b) make it so that the Host has to click the "Next Question" box to manually advance the game to "Get Ready", or (c) remove the correct answer highlighting and check box.